

DentalCare Pro — Implementation Prompts

Two separate, independent prompts. Prompt 1 fixes an existing sidebar bug. Prompt 2 restructures the onboarding tour into separate mobile/tablet and desktop tour files. Use each prompt on its own — do not run them together.

PROMPT 1 OF 2 — SIDEBAR RESPONSIVE FIX (sidebar.php)

Target file: sidebar.php

Type: Bug fix (independent of the tour restructure)

Context

sidebar.php contains a window resize listener, `handleResponsiveLayout()`, that auto-collapses the sidebar to icon-only width (w-20) whenever the viewport is between 768px and 1023px (tablet range). This listener fires on every resize event with no awareness of whether something else — specifically, the onboarding tour — has deliberately forced the sidebar open for a tour step. If a tour step expands the sidebar on a tablet-width viewport and a resize event fires (rotation, browser chrome change, etc.), `handleResponsiveLayout()` will immediately collapse it again, fighting the tour.

What to implement

Add a guard so `handleResponsiveLayout()` skips its auto-collapse behavior while the sidebar is being intentionally held open by another script (such as the tour).

Specific steps

1. Introduce a global flag, e.g. `window.sidebarForceOpenLock`, defaulting to `false/undefined`.
2. At the top of `handleResponsiveLayout()`, add an early return if `window.sidebarForceOpenLock` is `true` — skip the auto-collapse entirely while the lock is active.
3. Do NOT remove or alter the existing tablet auto-collapse behavior for normal (non-tour) usage — it should work exactly as it does today when no lock is set.
4. Keep the fix scoped to `sidebar.php` only. Do not modify `toggleSidebarCollapse()`, `_shiftContent()`, or the `DOMContentLoaded` restore-state block — those are unaffected and should not change.
5. The flag should be settable/clearable from outside `sidebar.php` (e.g. `window.sidebarForceOpenLock = true; ... = false;`), since the tour scripts (built separately) will be the ones setting it before expanding the sidebar and clearing it on step exit or tour cleanup.

Code reference — current function to modify

```
/* Auto-collapse sidebar on tablet widths natively */
function handleResponsiveLayout() {
  if (window.innerWidth >= 768 && window.innerWidth < 1024) {
    const sidebar = document.getElementById('mainSidebar');
    if (sidebar && !sidebar.classList.contains('sidebar-collapsed')) {
      sidebar.style.transition = 'none';
    }
  }
}
```

```
toggleSidebarCollapse();
requestAnimationFrame(() => { sidebar.style.transition = ''; });
    }
  }
}
```

Expected result

On tablet-width viewports, if `window.sidebarForceOpenLock` is true, resizing the window must NOT collapse the sidebar. With the lock false or unset, behavior is 100% unchanged from today. No visual or functional regression for desktop or mobile widths.

Acceptance checks

Resize browser to ~900px width (tablet range) with no lock set — sidebar still auto-collapses as it does today.

Set `window.sidebarForceOpenLock = true` in devtools console at ~900px, expand the sidebar manually, then trigger a resize event — sidebar must stay expanded.

Clear the lock (= false) and resize again — auto-collapse resumes normally.

Confirm no console errors and no change in behavior at <768px or >=1024px widths.

PROMPT 2 OF 2 — TOUR RESTRUCTURE: SEPARATE MOBILE/TABLET VS. DESKTOP FILES

Target: New /tour/ folder structure replacing portal-tour.js and dashboard-tour.js

Type: Architectural restructure (run only after Prompt 1 is complete and verified)

Context

The current tour engine (portal-tour.js) and its content config (dashboard-tour.js) use a single shared mechanic for all screen sizes: a floating tooltip that auto-places itself around a spotlighted element, with continuous scroll-tracking via requestAnimationFrame. On mobile and tablet widths this floating-tooltip approach is cramped and prone to layout fights with on-screen keyboards, address bar collapse, and small targets. Desktop/laptop should keep the existing floating spotlight concept exactly as-is. Mobile and tablet should share one new, separate tour concept: a fixed bottom-sheet tooltip with a one-time scroll-and-highlight instead of continuous tracking.

Mobile and tablet must use the SAME tour files (one shared concept, one set of files). Desktop and laptop must use a SEPARATE set of files (the existing concept, kept working as it does today). Any logic that is genuinely identical regardless of which concept is showing — keyboard handling, focus trap, skip/complete state, aria-hidden toggling, collapsible-nav expand/collapse step transitions, and the dashboard mock-data injection itself — must be pulled out into shared, global-use files rather than duplicated.

Breakpoint rule (use consistently everywhere)

Less than 768px wide → mobile sub-case (off-canvas drawer + hamburger nav)

768px to 1023px wide → tablet sub-case (icon-only collapsed sidebar, no hamburger)

1024px and wider → desktop/laptop concept

Mobile and tablet sub-cases (anything under 1024px) both load the SAME compact tour files. Only desktop/laptop (1024px+) loads the separate desktop tour files.

Required folder structure

```
/tour/  
portal-tour-core.js           (shared engine base – global use)  
dashboard-tour-shared.js     (shared mock-data + cleanup logic – global use)  
  
mobile-tablet/  
portal-tour-compact.js       (extends core – bottom-sheet concept)  
dashboard-tour-compact.js    (step definitions for mobile + tablet)  
  
desktop-laptop/  
portal-tour-desktop.js       (extends core – floating spotlight concept)  
dashboard-tour.js            (step definitions for desktop/laptop only)
```

1. portal-tour-core.js — shared engine base (global use)

Create this as a base class, PortalTourCore, containing everything that does not change between the two visual concepts. Both portal-tour-compact.js and portal-tour-desktop.js will extend this class.

Move into this file, unchanged in behavior:

`_injectStyles()` — the base animation/transition CSS (easing curves, fade classes). Keep the existing timing values exactly.

`handleKeyDown()` — Escape key closes the tour via `skip()`.

`trapFocus()` — Tab/Shift+Tab focus trapping inside the tooltip/sheet.

`skip()`, `complete()`, and the `dcpro_tour_completed` localStorage flag.

aria-hidden toggling on `#contentContainer`, `#mainSidebar`, and `#header` when the tour starts and ends.

The collapsibleNav step-transition logic currently in `renderStep()` — checking `activeNavCollapse`, calling `expand()/collapse()` between steps, and the 350ms wait.

Step navigation control flow: advancing `currentStep`, building/updating the dot indicators, and wiring the next/back/skip button click handlers (the parts of `buildTooltipHTML` and `renderStep` that are about WHICH step to show next, not how it's visually rendered).

The `onCleanup` external callback invocation contract — same signature, same timing (called at the end of `cleanup()` with the last `currentStep`).

Leave as abstract / overridable in subclasses:

`createElements()` — markup differs (floating card vs. fixed sheet).

`positionTooltip()` — auto-placement cascade vs. fixed bottom-anchor.

`trackTarget()` — continuous rAF-follow (desktop) vs. one-time highlight-and-stop (compact).

Any `scrollIntoView()` block/offset behavior tied to tooltip height.

Important: when forcing open a collapsed nav via `collapsibleNav.expand()` at tablet widths, set `window.sidebarForceOpenLock = true` (added in Prompt 1) before calling `expand()`, and set it back to false immediately after `collapse()` is called or during `cleanup()`. This prevents `sidebar.php`'s tablet auto-collapse listener from fighting the tour. Do this inside the shared core logic so both concepts benefit automatically — though only the compact concept will encounter tablet widths in practice.

2. dashboard-tour-shared.js — shared content logic (global use)

Pull the parts of the current `dashboard-tour.js` that are pure data/business logic and do not depend on which visual concept is displaying them:

`injectDashboardMockData()` — the full mock-data injection function, unchanged (stat counters, hero card, visit nudge, timeline HTML, preview badge element creation).

`ensureMockViewActive()` — the populated-area visibility check.

The post-tour restoration branch currently in `onCleanup` and the final step's `onEnter` — the `isNewPatient` check that decides whether to show the empty state or call `window.loadDashboardData()`. Expose this as a shared helper function both step-definition files can call.

Both `dashboard-tour-compact.js` and `dashboard-tour.js` should import/reference these shared helpers rather than redefining them.

3. mobile-tablet/portal-tour-compact.js

Create PortalTourCompact extends PortalTourCore, implementing the new bottom-sheet concept for any viewport under 1024px wide.

createElements(): render the tooltip as a fixed, bottom-anchored sheet (full-width on phone widths, a centered narrower sheet on tablet widths — a single CSS breakpoint inside this file, not a separate file). Rounded top corners only, safe-area-inset-bottom padding, progress dots at the top of the sheet, content stacked vertically below.

positionTooltip(): replace the auto-placement cascade entirely. The sheet always docks to the bottom of the viewport. No top/left calculation needed beyond centering width on tablet.

trackTarget(): replace continuous rAF tracking with a one-time calculation. On renderStep, scroll the target into view once using block: 'center' with a vertical offset that accounts for the sheet's height (so the target lands centered in the visible area ABOVE the sheet, not behind it). Compute and apply the highlight rectangle once; do not re-run on every animation frame.

For steps with no target (e.g. the Welcome step), dim the background behind the sheet without a circular full-screen spotlight — just show the sheet with a simple dimmed backdrop.

Add swipe gesture support on the sheet: a left swipe advances to next (equivalent to clicking Next), a right swipe goes back (equivalent to clicking Back). Implement via touchstart/touchend coordinate delta on the sheet element, in addition to keeping the existing Next/Back/Skip buttons fully functional.

Within this single file, branch on window.innerWidth < 768 only where mobile and tablet genuinely need different targets (this happens in the step-definition file below, not necessarily in the engine itself, but the engine must support per-step target functions returning different elements based on width, same way the current code already does).

4. mobile-tablet/dashboard-tour-compact.js

Create the step definitions object (e.g. dashboardTourConfigCompact) for the compact concept, reusing dashboard-tour-shared.js helpers for data injection and cleanup.

Steps 1-3 (Welcome, The Hub, Staying on Track): keep equivalent targets/copy to the current dashboard-tour.js, calling the shared injectDashboardMockData() / ensureMockViewActive() helpers on enter.

Step 4 (Explore More) must branch within this file based on width: under 768px, target #mobileMenuBtn with copy about tapping the menu icon; from 768px to 1023px, target the icon-only sidebar directly (#mainSidebar.space-y-1) with copy that matches an icon-only nav (no visible text labels) rather than the desktop copy that references a labeled menu.

Step 4 must still use the collapsibleNav mechanism (isCollapsed/expand/collapse tied to toggleSidebarCollapse) so the core's forced-open lock logic from portal-tour-core.js engages correctly at tablet widths.

Step 5 (Taking Action): keep equivalent logic to the current final step, using the shared isNewPatient helper from dashboard-tour-shared.js instead of inlining the branch.

onCleanup: call the shared restoration helper from dashboard-tour-shared.js instead of duplicating the isNewPatient branch.

5. desktop-laptop/portal-tour-desktop.js

Create PortalTourDesktop extends PortalTourCore, carrying over the EXISTING floating-tooltip behavior from the current portal-tour.js with no functional changes:

createElements(): same floating card markup/classes as today (w-[340px], shadow-2xl, rounded-2xl, etc.).

positionTooltip(): keep the existing bottom→top→right→left auto-placement cascade and safe-center fallback exactly as implemented today.

trackTarget(): keep the existing continuous requestAnimationFrame scroll-tracking exactly as implemented today.

This file should only ever be instantiated for viewports 1024px and wider — confirm the trigger code (wherever "Start Tour" is clicked) checks window.innerWidth >= 1024 before instantiating this class.

6. desktop-laptop/dashboard-tour.js

Trim the current dashboard-tour.js down to ONLY the desktop step definitions:

Remove the mobile-only branching currently inside step 4's text/target functions (the window.matchMedia('(min-width: 768px)') checks) — this file now only ever runs at 1024px+, so step 4 can target the expanded sidebar directly with no width branching at all.

Replace the inline injectDashboardMockData(), ensureMockViewActive(), and the isNewPatient cleanup/onEnter branches with calls to the shared helpers from dashboard-tour-shared.js, same as the compact file.

Keep all other step copy, targets, and timing identical to the current file.

7. Loading / instantiation logic

Wherever the tour is currently started (the "Start Tour" trigger), update it to:

Load all four script files normally (portal-tour-core.js, dashboard-tour-shared.js, both concept pairs) via standard <script> tags — no dynamic import needed for this local XAMPP environment.

At the moment the trigger fires (not on page load), check window.innerWidth: if less than 1024, instantiate PortalTourCompact with dashboardTourConfigCompact; if 1024 or greater, instantiate PortalTourDesktop with the existing dashboardTourConfig from dashboard-tour.js.

Do not re-check the width mid-tour and switch concepts — decide once at trigger time, consistent with how the rest of the app already treats resize transitions during an active session.

Acceptance checks

At >=1024px: tour behaves pixel-for-pixel identically to the current implementation (floating tooltip, auto-placement, continuous tracking).

At <768px: tour shows a bottom-anchored sheet, one-time highlight (no jitter on scroll), swipe and button navigation both work, step 4 targets the hamburger menu.

At 768-1023px: tour shows the same bottom-sheet concept as mobile (centered, narrower), step 4 correctly force-expands the icon-only sidebar using the Prompt 1 lock flag, and a window resize during that step does

not collapse it back.

No duplicated logic: confirm keyboard handling, focus trap, skip/complete, and mock-data injection exist in exactly one place each (the shared files), not copied into both concept folders.

Confirm dashboard.php loads the correct four files and the trigger code's width check is in place.